Digital Wallet Coding Challenge

The Challenge

You're writing an online wallet application and the goal is simple: Receive and send money.

Receiving Money

When a client receives money, the bank will notify us with a webhook with a body format the bank controls. The webhook can contain multiple transactions in a single call. On each webhook, the app should parse the transaction line and import it into the database, attaching it to the client in the process. An important thing to keep in mind is that the bank will sometimes report the same transaction twice or more, which should have no effect on the final state of a client's transactions list. Also, the app must be able to stop processing webhooks (ingestion) without dropping the incoming webhooks.

Sample Webhook From PayTech Bank

Each line is a transaction. The format is: Date, Amount (two decimals), "#", Reference, "#", Key-value pairs where the Key is before the / and the value is after it.

```
20250615156,50#202506159000001#note/debt payment march/internal_reference/A462JE81
```

Sample Webhook From Acme Bank

Each line is a transaction. The format is: Amount (two decimals), "//", Reference, "//", Date

```
156,50//202506159000001//20250615
```

Sending Money

All banks will use a standard XML format to communicate the intent to transfer money. Based on the request details, you should build the XML accordingly. Here's a sample of how the XML could look:

```
<?xml version="1.0" encoding="utf-8"?>

<PaymentRequestMessage>

 <TransferInfo>

 <Reference>e0f4763d-28ea-42d4-ac1c-c4013c242105</Reference>
```

```
<Date>2025-02-25 06:33:00+03</Date>

<Amount>177.39</Amount>

<Currency>SAR</Currency>

</TransferInfo>

<SenderInfo>

<AccountNumber>SA6980000204608016212908</AccountNumber>

</SenderInfo>

<ReceiverInfo>

<BankCode>FDCSSARI</BankCode>

<AccountNumber>SA6980000204608016211111</AccountNumber>

<BeneficiaryName>Jane Doe</BeneficiaryName>

</ReceiverInfo>

<Notes>

<Note>Lorem Epsum</Note>

<Note>Dolor Sit Amet</Note>

</Notes>

<PaymentType>421</PaymentType>

<ChargeDetails>RB</ChargeDetails>

</PaymentRequestMessage>
```

A Couple of Notes

- The **Notes** tag must not be present if there are no notes.
- The **PaymentType** tag must only be present if its value is other than 99.
- The **ChargeDetails** tag must only be present if its value is other than SHA.

For this, you need to only worry about generating the XML. Communication with the bank or tracking the transfer in the database are out of scope for this part of the challenge.

Guidelines

- Please include a commit with the framework's init files, and then organize your work in commits as you see fit, so your work is not contaminated by the framework files.
- Using Laravel is encouraged but not mandatory.
- Submitted code must look clean with standard styling applied. This includes consistent indentation, no commented-out code or debug calls like `dd()` or `dump()`.
- Adding documentation in the README file is heavily encouraged. You can use this space to elaborate on design decisions or considerations.
- Feel free to add code comments to communicate with the reviewer in places where you think it's beneficial.
- It is acceptable to over-engineer the solution to demonstrate skills.
- Ensure your solution is covered with automated tests.
- Take efficiency and scaling into consideration. An example would be to create a test case that parses a webhook with 1,000 transactions inside of it with acceptable performance.